The Problem:

Buying anything online **needs a lot of research** to get the best worth for money. This consumes a **lot of time**. With the vast variety of products available online and e-commerce websites, many a times a lot of cross checking is involved. For certain technical products, specifications also need to be compared. Sometimes certain additional constraints (like delivery time) also need to be met. While one can ask a general-purpose AI to compare items, this may not work well since the AI doesn't have access to all the specific information and explaining it the context again wastes time. This is the motivation for creating a fast, easy-to-use E-commerce assistant which does the research job for you.

The Solution:

An **LLM based e-commerce analysis agent** which performs all the research about the products across various websites and their credibility and shortlists a bunch of **budget-friendly**, **fast-delivery good-quality** trustable candidates, out of which the user can choose what they want, with further conversation with the LLM if required. The current focus is primarily on online shopping but, if possible, it will be **extended to food delivery, ride-booking, grocery delivery etc.**

Existing solutions:

Since it's not a very unique idea, **some solutions already exist.** There are some apps that allow the user to compare products across different platforms, but they **do not use any AI** to do the research. There are some AI based tools which make comparisons, but there are a few shortcomings. But these tools do it **only for online shopping**, as opposed to the versatility aimed in this project.

With a normal LLM, all data is processed with equal importance and due to the vastness of the data, it can lead to 'lost in the middle' issues, hallucinations or the LLM may not do enough research. On the contrary, this prospective solution compares every parameter individually, enhancing accuracy and stores the evaluations programmatically eliminating hallucinations. Also, different weightage is given to different parameters.

Why a 'local' LLM:

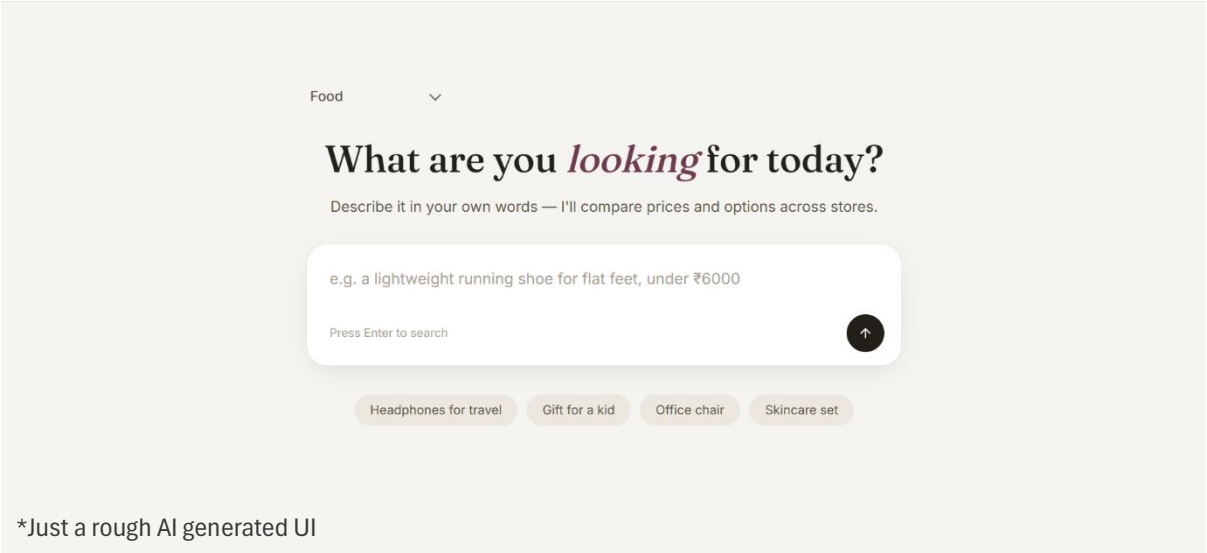
- It can use private information like bank offers and coupon codes to find the best deal and can also handle delivery date constraints.
- It can access category specific info which is only obtainable via scraping, for example availability of cabs (in case of transport) or certain combo offers.
- It can keep track of important personal long term info (for example size in case of clothing) and user preferences (especially dietary preferences in case of food ordering).
- It cuts token tax for a cloud based LLM. Also, it can give better context to the LLM which is useful when it is called so many times.
- It can also use order history for preferences or compatibility (in case of technical devices).

User Interface:

The user will use it through a web application or a mobile app.

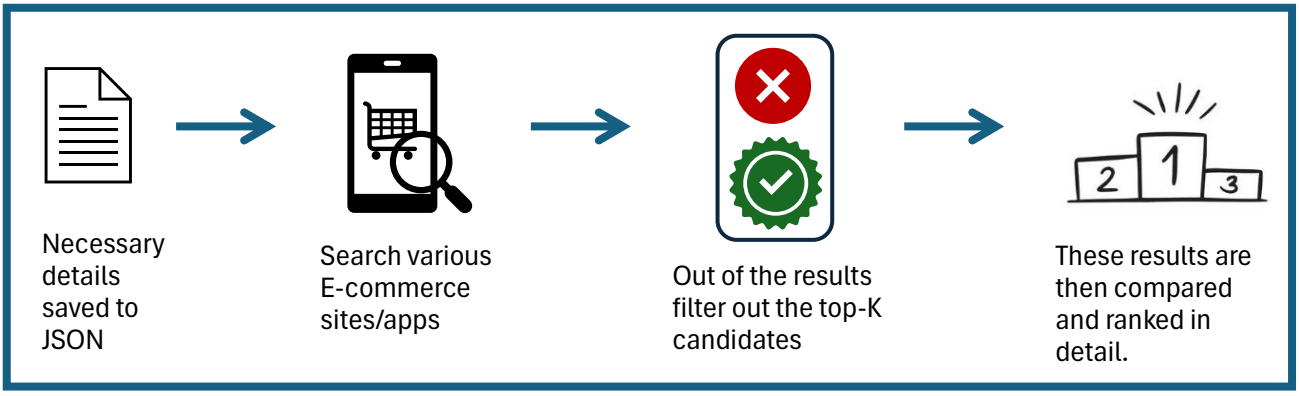
- The user opens the application and selects the category of service (online shopping, food delivery, transportation, etc.) each of which is associated with a set of possible apps (the user can add or remove apps to this based on their preference).
- Then the user describes what they want, with all the necessary details. They can select certain filters (like a slider for budget).
- The AI then reads all the info and does all the research it wants to and outputs a bunch of assorted options which are best for the user, along with short descriptions.
- The user can further ask the AI for recommendations or questions.
- A comparison button is provided to view the head-to-head comparison between products.
- The user then clicks the option they like, and it redirects them to the respective site/application where they can complete the purchase.
- Later, a Google Chrome extension can also be made for users to easily compare a bunch of selected products.

Implementation plan:



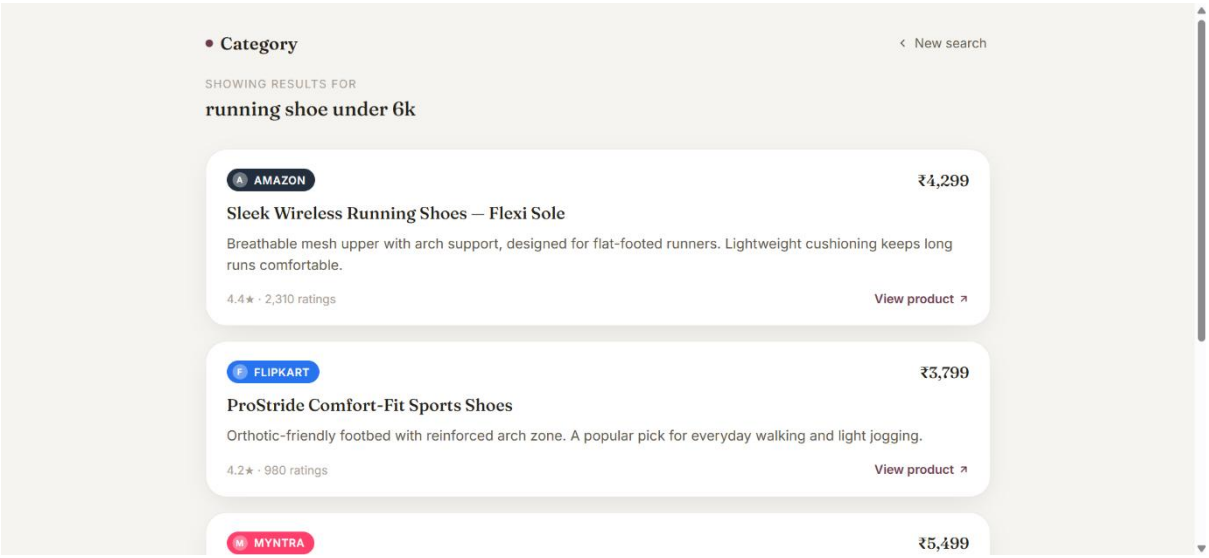
*Just a rough AI generated UI

User selects a category and describes his needs in the text box



Backend processing

The results are ordered with the most fitting deal at the top.



*Just a rough AI generated UI

Revised (and detailed) implementation architecture:

As mentioned earlier, the user input will be through a web/local app. First the user selects a category – general shopping, transport, food, etc. Then through various input formats, it takes in the necessary information and saves it to a local JSON file and calls the python backend.

1. Query generation and programmatic filtering:

- The LLM is given the primary constraints mentioned in the prompt, user persona and other required details and it is asked to generate a search query for the E-commerce sites/apps along with a list of filters (eg budget).
- This query is used to search for the possible products in the set of service providers (like amazon) mentioned in the category. The products are filtered either using the filters provided by the app or by using an automated script.

2. Top-K selection:

After the first level of filtering is done, each of the options is evaluated and a multi-dimensional vector is generated for each option, in the following format -

```
(star_rating, -cost, discount%, -cost_above_average, LLM_rating, -delivery_time, bank_or_combo_offers, other_parameters…)
```

The objective is to '**maximize'** this vector. The various components are –

- Star Rating: The overall customer rating of the product displayed on the site.
- Cost: The Z-score normalized final cost of the product (after all offers). It is negated since cost needs to be minimized.
- Discount: The maximum discount on the original price (if any) as a percentage **including bank offers**.
- Cost above average: The difference between the listed cost and the usual cost of the product/service predicted by LLM. (negated to minimize)
- LLM rating: The LLM **analyzes the overall product description and a few of the customer reviews** (maybe like best 3 and worst 2) and is asked to rate it on a scale of 1 to 100. It is a lightweight (maybe using RAG) analysis since it has to be performed on each of the many products.
- Delivery time: Mostly for transport, groceries or food delivery where time matters. In other cases also it can be used if a delivery constraint is specified. (negated to minimize)
- Bank/Combo offers: The best bank offer or a combo deal for the specific product.
- Other product specific parameters: The LLM is asked to generate some (<10) of the most **important specifications that need to be considered** while buying the product (from internet opinion via RAG). The values of these specifications are then scraped programmatically and tabulated.

Once the vectors are generated for each product, they are then multiplied by a hardcoded weights vector that determines the relative importance of each of the above fields. Then all these vectors are ranked using **a topsis ranking system**. Out of this ranking, the top-K are selected to the next stage.

3. Thorough analysis and sorting of selected products:

Specification-wise rating:

- The LLM is asked to generate all the parameters that can be compared while buying the required product (from internet opinion via RAG) and their relative importance weights.
- One of the K products (maybe topmost) is taken as the baseline. All other products are compared with respect to this baseline across the various parameters.
- For each product, the LLM classifies each parameter into {"significantly worse", "worse", "equal", "better", "significantly better"} in comparison to the baseline.
- Here, the local user persona also comes into play as the parameters are given priority based on what the user prioritizes (eg some people mainly care about budget, some care about technical aspects, some care about design etc.)
- The most significant results are then compiled into a short textual 'product summary'.
- The head-to-head table is saved for presenting to user.

Customer review analysis:

- The LLM ingests all the customer reviews of that product one-by-one and stores a summary of the reviews. This is the 'reviews summary'.
- For processing a review, the LLM first analyses whether it is genuine or just a fake review. If genuine, it extracts the key points mentioned in the review.
- Then these points are added to the summary. The summary consists of the key points and their frequencies.
- While adding a new point, if there's an existing point that conveys the same thing (identified maybe by calculating raw semantic distance) then its frequency is increased or else a fresh point is added.
- If available, even reddit or other reviews about that specific company and product are also analyzed.

Now the products are presented in a pairwise manner to the LLM with their product summary and review summary which then decides which product is better out of the two, for the user persona. In this way, the products compete against each other in a knockout fashion and the options are finally ranked from best to worst and a short summary is generated – brief product details, why it is the best deal, possible offers, pros and cons.

4. Output:

- All this output is saved as an XML and after finishing, the backend returns back to the frontend which parses the XML and neatly displays the data.
- Any further questions are directly relayed to the LLM with the required context.
- Each option is linked to the respective site/app where it can be purchased.

LLM model: LLM model used will be either Qwen-3.5-2B or Gemma-4-E2B, focusing on mainly speed (for reading so much data) and good reasoning skills (to generate credible scores).

Chrome extension: An additional chrome extension can be added when the user has already selected few prospective products and wants to compare them thoroughly. In this case, the stage 3 is called directly by the extension and the comparison table is also displayed to the user.

Opportunities for optimization (not much idea about this):

- Fine-tuned LLM for comparison. It can also be fine-tuned per service.
- GPU optimizations to read many sites simultaneously.
- Simultaneous processing of mutliple LLM prompts.
- Specialized memory structures for storing and comparing data.

Challenges:

- This will need a HUGE number of tokens (mainly for reading info etc.). And given that the speed of a normal LLM on a local GPU will be barely around 50 tokens/sec, it **might take very long** to process. Moreover, the **LLM needs to be called many times**.
- Reading from a website is easy **but accessing data from apps is much harder**, for services like transport, food delivery etc. Certain APIs might be required, and they are not easy to obtain.
- **Robot protections** in the websites/apps can hinder information gathering.
- There's **no pre-existing data for fine-tuning**. Since the problems are mainly based on situations, it is hard to create/obtain data for training.